Group 9: DART - Diabetes Analytics & Research Team

CS 133 Sec 01 Group 9

Topic: Predictions Based on Diabetes Health Indicators

Anh Tran, Willy Tang, Jacob Atanacio, Aziza Mohammed

Table of Contents

1. Abstract 3

2. Background 4

2.1 Defining Problems 5

3. Data Visualization Strategy & Interactive Component 5

3.1 Static Visualizations 5

3.2 Interactive Visualization 6

4. Machine Learning Pipeline & Evaluation 8

4.1 Data Preparation and Pipeline 9

4.1.1 Training/Test Split 9

4.2 Model Training 11

4.3 Model Evaluation & Testing 11

5. Results, Code Output & Interpretation 14

5.1 Machine Learning Pipeline Pre-Processing Filtering 15

5.2 Data Distribution Highlights 15

5.3 Selection of the Best ML Model: Random Forest 15

5.4 Metric Handling for Three-Class Interpretations 16

6. Reproducibility, Github, & Documentation 16

6.1 GitHub 17

6.2 Reproduce 17

6.3 Documentation 17

1 Abstract

Our project explored how factors such as socioeconomics, mental health, BMI, and diet played a role in influencing diabetes risk. We worked with our data to determine whether our three machine learning models could be trained to make predictions on whether an individual had diabetes. Our goal was to train our three models to achieve over 80 percent in accuracy, precision, and recall while addressing overfitting and underfitting. Additionally, we sought to determine what factors within our dataset could be used to determine the diabetes risk. We used k-nearest neighbors (KNN), random forest, and logistic regression as our models.

We used the Diabetes Health Indicators Dataset from Kaggle as our main dataset, which contained 100,000 rows of preprocessed synthetic data and with 31 columns of features (Thalla). It contained features about lifestyle, demographics and medical values such as physical activity minutes per week, education level, and systolic blood pressure (Thalla). Our dataset contained imbalanced number of diabetes diagnoses with a 60:40 split of diabetes and no diabetes. For diabetes stage, we observed Type 2 representing approximately 60 percent of our rows, with 32 percent representing pre-diabetes, and the remaining 8 percent representing no diabetes, Type 2 diabetes, and gestational diabetes (Thalla). We additionally incorporated the Centers for Disease Control and Prevention's (CDC) 2015 Behavioral Risk Factor Surveillance System (BRFSS), which assisted in addressing a question posed for our data exploration regarding mental health and socioeconomic factors. This dataset was preprocessed and cleaned of missing values by the Kaggle user who uploaded the dataset (Teboul). It contains over 250,000 rows of data with 21 columns and features such as self-reported mental health, if individuals avoided going to doctors due to costs, and smoker status (Teboul). It contains approximately 210,000 rows of data with no diabetes diagnosed, 4,000 rows with prediabetes, and 35,000 rows of diabetes.

Our visualization strategy was to utilize different plots to add variety to our visual information. We utilized hex plots, scatter plots, box plots, violin plots, heatmaps, and an interactive plot to portray different aspects of information. Heatmaps were used to visually demonstrate the prevalence of diabetes with two feature columns and their different values. Violin plots and box plots were used on a categorical data column and numeric column to show the statistical distribution of data. In particular, box plots helped to compare two different sets of box plots next to each other and show the differences in distribution. Violin plots helped to compare differences in density of data and how it may vary from one categorical value to another. Hex plots conveyed the bell curve distribution and showed how data was clustered. Scatter plots allow us to visually represent how singular data points were distributed relative to the diabetes diagnosis an individual had. Color was utilized in scatterplots and heatmaps. Heatmaps utilized color to convey prevalence, with some using darkness to indicate prevalence while others used

color to distinguish between two ranges of values. In scatterplots, it helped to distinguish between clusters of points plotted.

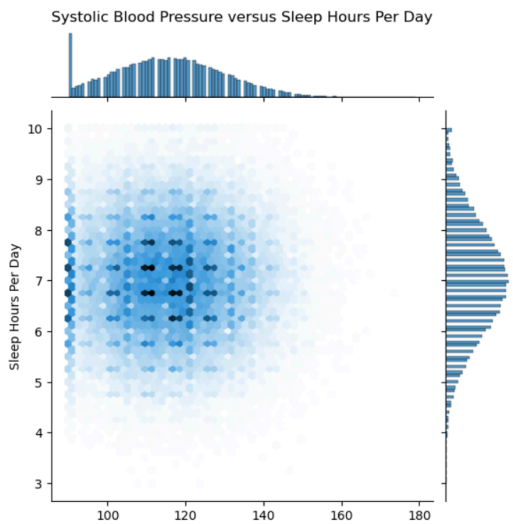
Major insights we found were that our dataset was synthetic and that it was designed based on "statistical distributions and correlations derived from real-world medical research studies" (Thalla). Many of our columns that did not contain binary values tended to follow the distribution of either a bell curve or skewed bell curve distribution. A limitation of our data is that it may not fully accurately portray real world data distribution due to a lack of data variance. Our logistic regression model achieved a mean accuracy of 85 percent, weighted recall mean of 85 percent, and weighted precision mean of 97 percent. The KNN model achieved a mean accuracy of 94 percent, weighted recall mean of 94 percent, and weighted precision mean of 94 percent. Our random forest model achieved a mean accuracy of 92 percent, weighted recall mean of 92, and weighted precision of 93 percent. Also, removing our diagnosed_diabetes column assisted in addressing data leakage and overfitting that occurred during the training of our third model, the random forest.

2 Background/Problem definition

Diabetes is a disease affects over 500 million people across the world as of 2021 (Tasin et. al). The pancreas becomes unable to regulate blood sugar due to its inability to produce insulin (Tasin et. al). Diabetes can cause damage to an individual's body. For example, with high blood sugar, "On physical examination of someone with hyperglycemia, poor skin turgor (from dehydration) and a distinctive fruity odor of their breath (in patients with ketosis) may be present" (Sapra & Bhandari). An individual's skin can lose its elasticity. The effects on the body can become severe, and "People with diabetes for a long time can get several complications like heart disorder, kidney disease, nerve damage, diabetic retinopathy etc. But its risk can be reduced if it is predicted early" (Tasin et. al).

Identifying diabetes can be challenging, as "Early and accurate diagnosis of diabetes mellitus, especially during its initial development, is challenging for medical professionals. Artificial intelligence and machine learning techniques, providing a reference, can help them gain preliminary knowledge about this disease and reduce their workload accordingly." (Tasin et. al). Machine learning can assist in the prediction of diabetes and reduce risks if left undiagnosed, as it is not easy to identify the early signs of diabetes. It has the potential to ease the challenge of identifying diabetes while also being able to act sooner to potential damage to the body diabetes can inflict. For example, researchers utilized machine learning on the Pima Indian Dataset to produce predictions with XGBoost and ADASYN approach, and it achieved an accuracy of 81 percent (Tasin et.al).

For visualization, we noticed that our dataset tended to have several columns with distributions representing a variation of the bell curve distribution. Some columns skewed towards one side of the range of data or the other. Other columns had a bell curve-like distribution. For example, when plotting systolic_bp (systolic blood pressure) against sleep_hours_per_day, we noticed that our hex plot tended to cluster towards a point in a circular pattern.



Our data tended to learn towards a bell curve distribution, but we decided to further refine it with StandardScaler to address possible fitting issues on since certain numeric columns that had skewed bell curve distributions. Additionally, it would reduce the wide range of values from certain columns such as triglycerides. Furthermore, we needed to use OneHotEncoder to adjust our categorical data that had strings of values and convert it into numeric data that our models could process.

2.1 Defining Problems

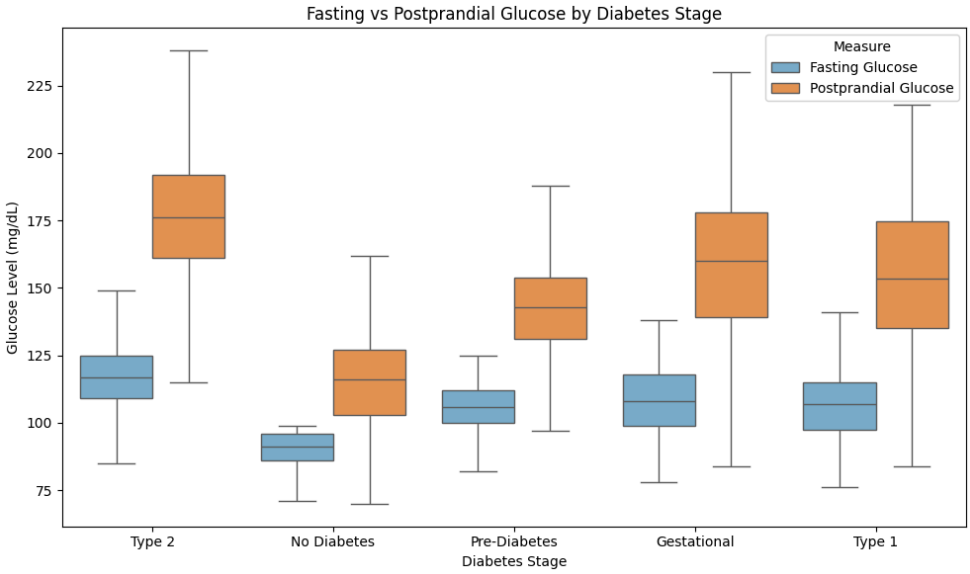
- 1. How do lifestyle factors such as physical activity, diet, and BMI influence the likelihood of having diabetes?
- 2. Are there differences in measuring fasting glucose and prandial glucose for determining blood sugar levels, a factor considered for diabetes?
- 3. How accurately does a person’s self-reported "General Health" match up with their actual BMI and Blood Pressure?
- 4. Which profiles express a higher risk to diabetes profiles consist of a combination of indicators(male, smoker, high school education, etc…)

- 5. How does the combination of socioeconomic barriers and mental health influence the prevalence and management of diabetes?

3 Data Visualization Strategy & Interactive Component

3.1 Static Visualizations

To answer question 2:

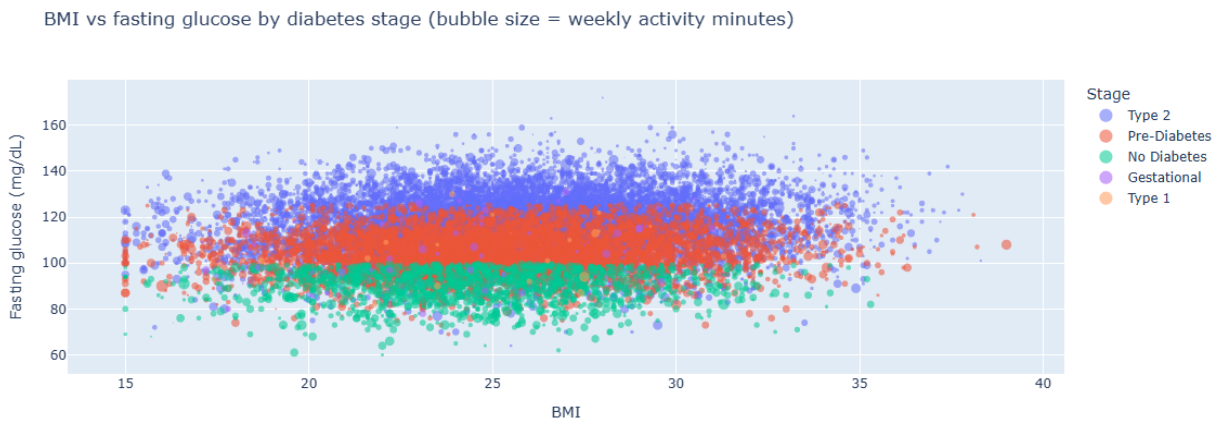


Based on the box plot generated from the code snippet above, we are able to observe the different measurements (in mg/dL) of fasting glucose and postprandial glucose across the five stages of diabetes. Furthermore, when referring to an article from the Mayo Clinic, we are able to compare the averages of each value in the plot.

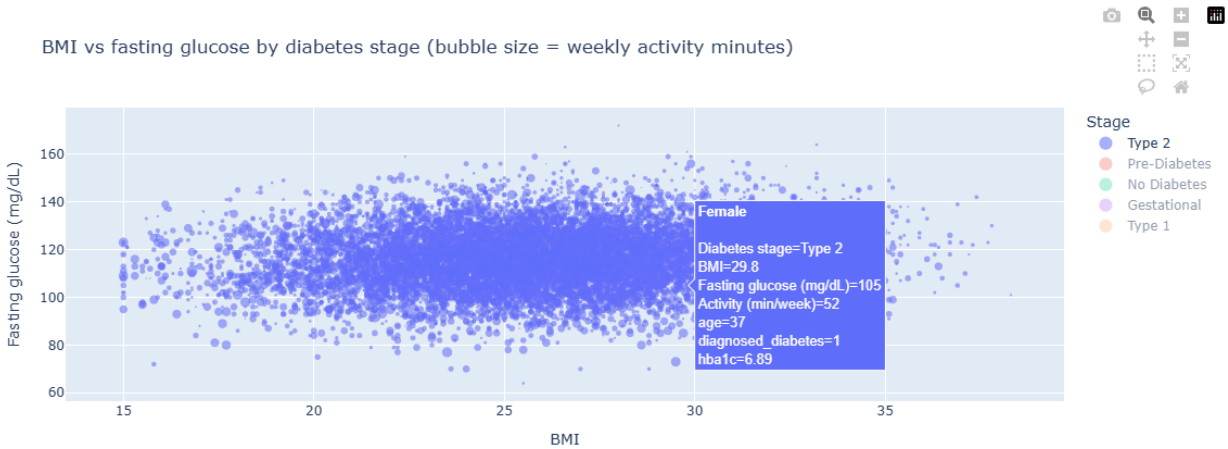
For instance, in our plot, individuals without diabetes show fasting glucose levels below 100mg/dL, which corresponds with the article's statement that "a fasting blood sugar level less than 100 mg/dL (5.6 mmol/L) is normal"(Mayo Clinic, n.d.). In addition, for individuals with prediabetes, the plot displays fasting glucose levels between 100 and 125 mg/dL, which aligns with the Mayo Clinic's claim that "a fasting blood sugar level from 100 to 125 mg/dL (5.6 to 6.9 mmol/L) is considered prediabetes"(Mayo Clinic, n.d.).

3.2 Interactive Visualization

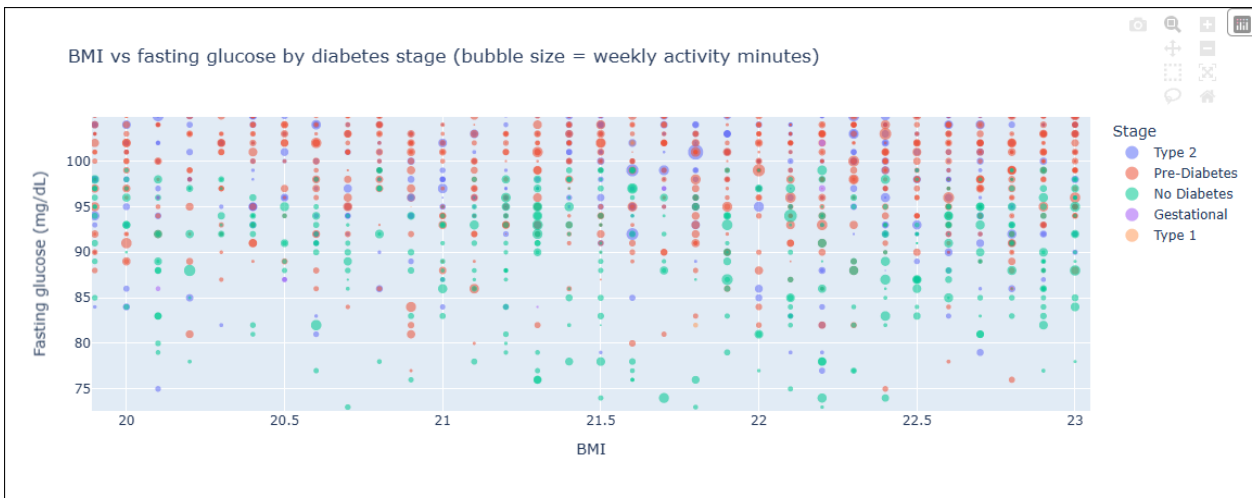
Aside from the initial five questions we came up with, we wanted our interactive plot to analyze how BMI and fasting glucose are related, and how diabetes stages differ in that space. In order to achieve this, we set BMI as the x-value and fasting glucose as the y-value, where the size of a data point is dependent on the weekly activity minutes each individual is active for.



The interaction allows us to hover over each data point to show the diabetes stage, BMI, age, HbA1c, diagnosis condition, fasting glucose level, and activity level, which helps us analyze outliers and clusters without overcrowding the chart. Additionally, the legend on the right lets us click on different diabetes stages to turn them on and off, making it easier to see any overlap or separation between groups. The zoom feature allows us to focus on smaller, dense regions for closer analysis. When we combine the pan function while being zoomed in, it lets us move around the zoomed-in area to explore more data points. Since the dataset is very large (100k rows, even after sampling 20k), these interactions are important because the scatter plot would be too dense to interpret clearly without interactability, which makes it much harder to analyze patterns between BMI and fasting glucose across diabetes stages.



The above image is from deselecting all the diabetes stages except for Type 2 in the legend table while hovering over a random point to view information regarding an individual.



By zooming into a random section of the scatter plot, we can analyze the distribution of data points within a specific region between the different stages of diabetes, as seen in the image above. This is especially helpful for regions with high quantities of data points.

To run the interactive plot, you can clone our git repository and run the P3_interactive.ipynb. After running the code from the beginning, where you need to run the code block that import the required tools needed for graphing interactive plots to the code located in “Question 2: Create a simple interaction plot.” Once that is done, you will see the exact replica of the interactive scatter plot above and be able to interact with it.

4 Machine Learning Pipeline & Evaluation

4.1 Data Preparation and Pipeline

4.1.1 Training/Test Split

We implemented a stratified train-test split using StratifiedShuffleSplit with a distribution of 80% training set and 20% testing set. Since we are using "diabetes_stage" as the target variable, with the end goal of developing an ML model that can predict the stages of diabetes given an individual's information. Therefore, we used stratification so that the proportions of different diabetes classes are kept even between both the training and testing sets to prevent imbalances in datas.

```
# Dataset
diabetes_df = pd.read_csv('diabetes_dataset.csv')


kept_stages = ["Type 2", "Pre-Diabetes", "No Diabetes"]
diabetes_df = diabetes_df[diabetes_df["diabetes_stage"].isin(kept_stages)].copy()
diabetes_df = diabetes_df.reset_index(drop=True)

print("Filtered dataset shape:", diabetes_df.shape)
print(diabetes_df["diabetes_stage"].value_counts())

diabetes_df.head()
```

However, due to the very uneven distributions of datas between Type 2, Pre-Diabetes, No Diabetes, Type 1, and Gestational diabetes, we had to make a decision between whether or not we should train the model to identify Type 1 and Gestational diabetes. Since Type 1 and Gestational diabetes made up less than 0.5% of the datas in the dataset, even when combined, when we tried training the model on all 5 diabetes stages, all three models had very low cross-validation scores of 40-50% due to extreme imbalance between multiple classes. With such a low number of data points for the two types, the model was barely able to see those classes during training, causing it to struggle to learn any patterns. When we took out the two types, Logistic Regression has a mean score of .83, KNN with a score of .71, and Random Forest with a score of .91, which is the highest score of all three models

Therefore, we were stuck between two options due to the dataset's limitation: the first is to exclude Type 1 and Gestational diabetes from the prediction, which in turn will make the models much more accurate in predicting Type 2, Pre Diabetes, and No Diabetes. However, this would mean that Type 1 and Gestational diabetes will be outside of the scope for our models. The other option is to include all the types, which allows for a wider range of predictions across more classes, but it would significantly reduce the cross-validation scores of all our models by around 40-50%.

Since the project is related to human health, we had to go with the option that would provide the best prediction, even if it means having a smaller scope. Our understanding was that if this were used in real-world prediction, we would not want to include classes with very limited quantities of datas when training models, especially if we are doing predictions in the medical field, since giving wrong predictions regarding the health of patients can hurt them significantly, but you might also get into legal troubles.

4.1.2 Preprocessing & Pipeline

When preprocessing the datas, we defined "diabetes_stage" as the target variable and used all other variables as the features, besides the "diagnosed_diabetes" column, which we dropped from the training set. We had to drop this column because it was a numerical column with values 0 for unconfirmed diagnosed diabetes and 1 for confirmed diagnosed diabetes. Including this column would leak datas to the ML models because, since we are using only Type 2, No Diabetes, and Pre Diabetes, the models will learn that if the diagnosed_diabetes column contains a 1, then it must be Type 2. We then handled the numerical and categorical features separately, where we used SimpleImputer and StandardScaler to fill missing values and scale them accordingly. As for categorical features, we used SimpleImputer to fill missing values and OneHotEncoder to convert the variables into numerical format. Although our datasets had no missing values since the creator made it with ML in mind, we wanted to follow what was taught in the course's hands-on for good practices.

Additionally, we used Pipeline and ColumnTransformer, where Pipeline takes in the raw data from the datasets and applies the preprocessing steps we mentioned above to train the model. ColumnTransformer is used to make sure that the columns are being handled correctly between numerical and categorical columns.

```python
# Drop target and diagnossed diatebes columns from features
#Diagnose diabetes columns can isntantly tell the ML models that if diagnosed diabet
target_col = "diabetes_stage"
cols_to_drop = [target_col, "diagnosed_diabetes"]

X_train = strat_train_set.drop(columns=cols_to_drop, errors="ignore")
y_train = strat_train_set[target_col]
X_test = strat_test_set.drop(columns=cols_to_drop, errors="ignore")
y_test = strat_test_set[target_col]

# Split columns into numeric and categorical groups
num_attribs = X_train.select_dtypes(include="number").columns.tolist()
cat_attribs = X_train.select_dtypes(include=["object", "category"]).columns.tolist()

# Pipeline for numeric columns
num_pipeline = Pipeline(
    [
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ]
)

# Pipeline for categorical columns
cat_pipeline = Pipeline(
    [
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False)),
    ]
)

# Combine pipelines
full_pipeline = ColumnTransformer(
    [
        ("num", num_pipeline, num_attribs),
        ("cat", cat_pipeline, cat_attribs),
    ]
)

print("Target class distribution in training set:")
print(y_train.value_counts(normalize=True).round(4))
print("Numeric columns:", len(num_attribs))
print("Categorical columns:", len(cat_attribs))
```

4.2 Model Training

Logistic Regression:

- We used Logistic Regression as the first model since it is simpler and can serve as the baseline for our next two models, which uses the pipeline we created previously above. Additionally, we increased the iterations and had class weight to prevent class imbalance between the different classes of diabetes, which we talked about when deciding what classes to leave out. Afterward, the performance was evaluated with 5-fold cross-validation on the training set using metrics such as accuracy, F1-score, Recall, Precision, and ROC AUC.

K-Nearest Neighbors (KNN):

- KNN was implemented with the same preprocessing pipeline as the first model since we want to have all models be trained consistently. Since KNN uses the distance between points to determine neighbors, if one feature is too large and another is too small, then the small features will be

dominated by the larger features. Therefore, in the preprocessing step, we added StandardScaler in order to rescale the features so that they are on the same scale. Afterward, the model is evaluated with 5-fold cross-validation. We chose KNN for our second model because we wanted to see whether predicting diabetes stages based on similarity to nearby data points would perform better than predicting based on a relationship formula used by Logistic Regression.

Random Forest:

- Similar to the first two models, Random Forest was implemented with the same preprocessing pipeline. However, we chose RF since it is considered a stronger classification model than the first two we previously used, which is mainly due to its use of multiple decision trees. The model has a class weight to prevent imbalance in class, and we defined 300 for the number of decision trees it creates to reduce overfitting and make more accurate predictions. We also evaluated the model with 5-fold cross-validation

```
log_reg_pipeline = Pipeline([
    ("preprocess", full_pipeline),
    (
        "model",
        LogisticRegression(
            max_iter=2000,
            random_state=42,
            class_weight="balanced",
            solver="lbfgs",
        ),
    ),
])
scoring = ["f1_macro", "accura
scores = cross_validate(
    log_reg_pipeline,
    X_train,
    y_train,
    cv=5,
    scoring=scoring
```

```
knn_pipeline = Pipeline([
    ("preprocess", full_pipeline),
    ("model", KNeighborsClassifier(n_neighbors=15)),
])

knn_scores = cross_validate(
    knn_pipeline,
    X_train,
    y_train,
    cv=5,
    scoring=scoring
)
```

```
rf_pipeline = Pipeline([
    ("preprocess", full_pipeline),
    (
        "model",
        RandomForestClassifier(
            n_estimators=300,
            random_state=42,
            class_weight="balanced_subsample",
            n_jobs=-1,
        ),
    ),
])

rf_scores = cross_validate(
    rf_pipeline,
    X_train,
    y_train,
    cv=5,
    scoring=scoring
)
```

4.3 Model Evaluation & Testing

Scores after cross-validation of Logistic Regression, KNN, and RandomForest, respectively

Prediction of class: Type 2, Pre Diabetes, and No Diabetes.

```
Scores:
F1 Macro Mean:  0.833755039163778
Accuracy Mean:  0.8672063253012048
Recall Weighted Mean:  0.8672063253012048
Precision Weighted Mean:  0.8805148857592041
ROC AUC OVR Mean:  0.9614617496323392
```

```
Scores:
F1 Macro Mean:  0.7178847368962141
Accuracy Mean:  0.7883408634538152
Recall Weighted Mean:  0.7883408634538152
Precision Weighted Mean:  0.7911140373358522
ROC AUC OVR Mean:  0.8940888907716872
```

```
Scores:
F1 Macro Mean:  0.9182418834255323
Accuracy Mean:  0.920168172690763
Recall Weighted Mean:  0.920168172690763
Precision Weighted Mean:  0.9327837235270087
ROC AUC OVR Mean:  0.9738861637789686
```

Prediction of class: Type 2, Pre Diabetes, No Diabetes, and Type 1.

```
Scores:
F1 Macro Mean:  0.5727878548172923
Accuracy Mean:  0.727628202564888
Recall Weighted Mean:  0.727628202564888
Precision Weighted Mean:  0.8642885089823586
ROC AUC OVR Mean:  0.8848998965773243
```

```
Scores:
F1 Macro Mean:  0.5379191968941183
Accuracy Mean:  0.7867932151869537
Recall Weighted Mean:  0.7867932151869537
Precision Weighted Mean:  0.7887325728617449
ROC AUC OVR Mean:  0.7648599995077046
```

```
Scores:
F1 Macro Mean:  0.6874718560067044
Accuracy Mean:  0.9182220433990403
Recall Weighted Mean:  0.9182220433990403
Precision Weighted Mean:  0.929734125638763
ROC AUC OVR Mean:  0.8675003061802148
```

Prediction of class: Type 2, Pre Diabetes, and No Diabetes, Type 1, and Gestational.

```
Scores:
F1 Macro Mean:  0.4375851954135566
Accuracy Mean:  0.6500625
Recall Weighted Mean:  0.6500625
Precision Weighted Mean:  0.8556751296690053
ROC AUC OVR Mean:  0.8678016189175345
```

```
Scores:
F1 Macro Mean:  0.4295243116414775
Accuracy Mean:  0.784775
Recall Weighted Mean:  0.784775
Precision Weighted Mean:  0.7847061299864919
ROC AUC OVR Mean:  0.7088130463537015
```

```
Scores:
F1 Macro Mean:  0.5489672666987031
Accuracy Mean:  0.9150500000000001
Recall Weighted Mean:  0.9150500000000001
Precision Weighted Mean:  0.9236479659171802
ROC AUC OVR Mean:  0.8410835422681121
```

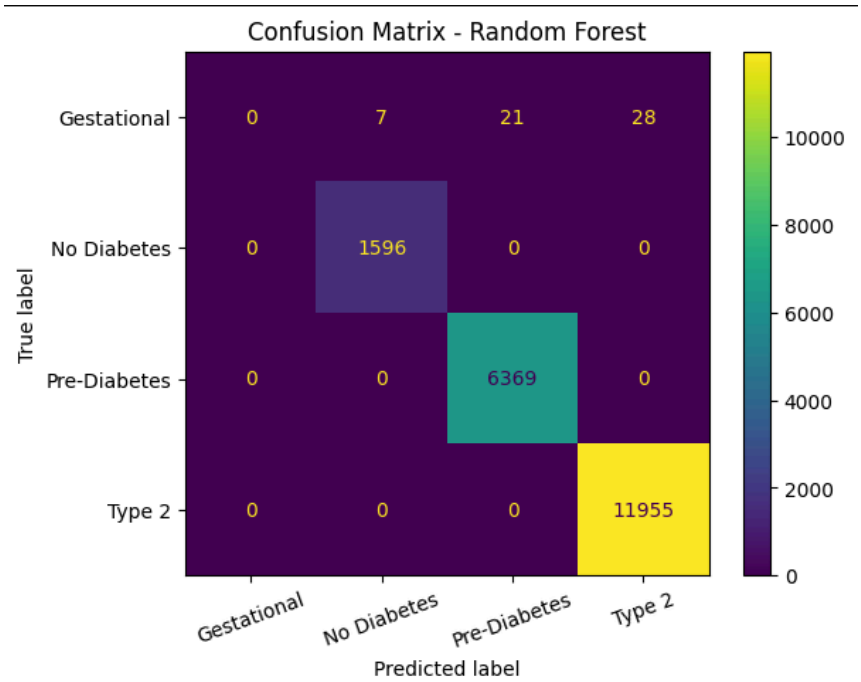
As you can see from the cross-validation scores after training of all 3 ML models across different numbers of classes, even though we are going with prediction on just Type 2, Pre Diabetes, and No Diabetes, you can see the distributions of how drastically the F1 macro mean score drops as we add in Type 1 and Gestational diabetes datas since the models are unable to perform well across all classes, especially for smaller ones.

Additionally, when looking at the KNN score for 3-class prediction, its score is at .71, which is must lower than Logistic Regression and Random Forest. We can make an assumption that is caused by regions of datas that are closely related to each other across different diabetes stages. For instance, if we look back at the interactive plot, the data points from individuals with Type 2 diabetes overlapped a lot with those of individuals who are Pre Diabetics. Therefore, KNN must have made the wrong predictions when looking at the nearest neighbors in those regions.

```
# Final testing with Random Forest on test set
rf_pipeline.fit(X_train, y_train)

y_pred = rf_pipeline.predict(X_test)
y_proba = rf_pipeline.predict_proba(X_test)

print("Model: Random Forest")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred, average="macro", zero_division=0))
print("Precision:", precision_score(y_test, y_pred, average="macro", zero_division=0))
print("F1:", f1_score(y_test, y_pred, average="macro", zero_division=0))
print("ROC AUC:", roc_auc_score(y_test, y_proba, multi_class="ovo"))


ConfusionMatrixDisplay.from_predictions(y_test, y_pred, xticks_rotation=20)
plt.title("Confusion Matrix - Random Forest")
plt.show()
```

```
Model: Random Forest
Accuracy: 0.9971966359631558
Recall: 0.75
Precision: 0.7475025398044993
F1: 0.7487490577042717
ROC AUC: 0.9260151866947997
```

After determining that Random Forest is the best-performing model, we tested it against the test set. The Random Forest model has a strong performance during cross-validation, with a high F-1 score and ROC AUC values, which means that it was able to learn well across the training folds. However, when we evaluated against a test set, the accuracy remained very high, while the F1-score decreased. This means that there is a possible imbalance between the classes, where the Random Forest model was able to perform well on classes with many data points but struggles with classes with fewer data points. If we take a look again at the data distribution, we can see that Type makes up nearly 60% of the data set while No Diabetes makes up only of 8%. This imbalance in datas most likely explains the lower F-1 score since the model has more difficulty correctly predicting underrepresented classes. Despite the lower F-1 score, the ROC AUC score is still high; therefore, the model is still able to distinguish between classes.



Based on the confusion matrix showing the performance of the Random Forest model, we can see that the model performs extremely well on the majority classes, such as No Diabetes, Pre- Diabetes, and Type 2. However, it fails entirely to correctly classify any instances of Gestational diabetes and appears to have misclassified them into other categories instead. This means that the model is biased toward the majority classes and struggles with minority classes, which relates back to what we mentioned above regarding the high overall accuracy but lower F1-score.

5 Results, Code Output & Interpretation

5.1 Machine Learning Pipeline Pre-Processing Filtering

In our P4_ML_Training_Testing notebook we ended up filtering the diabetes dataset to focus on three classes. These target variables within the class diabetes_stage consisted of:

1. **Type 2 - 59,774 rows**
2. **Pre-Diabetes - 31,845 rows**
3. **No Diabetes - 7,981 rows**

```
In [43]:
# Dataset
diabetes_df = pd.read_csv('diabetes_dataset.csv')


kept_stages = ["Type 2", "Pre-Diabetes", "No Diabetes"]
diabetes_df = diabetes_df[diabetes_df["diabetes_stage"].isin(kept_stages)].copy()
diabetes_df = diabetes_df.reset_index(drop=True)

print("Filtered dataset shape:", diabetes_df.shape)
print(diabetes_df["diabetes_stage"].value_counts())

diabetes_df.head()

Filtered dataset shape: (99600, 31)
diabetes_stage
Type 2          59774
Pre-Diabetes    31845
No Diabetes      7981
Name: count, dtype: int64
```

5.2 Data Distribution Highlights

We implemented a train-test split with **StratifiedShuffleSplit** with an 80/20 train:test ratio. Before we could route data into our pipeline, we executed data distribution cells to provide precise bounds of our patient profiles. The code output revealed the following distributions across the 100,000 rows:

- **Age -** The mean age found within our dataset was 50.12 years
- **Physical Health -** The mean BMI of the population was calculated exactly at 25.61 with a max of 39.2 Physical activity averaged at about 118,91 minutes per week.
- **Clinical Ranges -** Fasting glucose measurements ranged between 60 mg/dL to a maximum of 172 mg/dL with a mean of 111.11 mg/dL Postprandial glucose had a wider variance of 70 mg/dL to 287 mg/dL

5.3 Selection of the Best ML Model: Random Forest

We concluded that the highest performing model was the Random Forest algorithm for predicting our three-class target. While our Logistic regression model was underfit which caused its results to suffer from that and the KNN was computationally demanding, the Random Forest's architecture of multiple

decision trees enabled it to effectively capture the non-linear relationship between variables such as hbalc, insulin levels, and glucose fasting levels across the three stages.

5.4 Metric Handling for Three-Class Interpretations

When generating the final performance output for the Random forest model we made sure to retain our scoring metrics. While calculating the recall score, precision score, and the f1 score we explicitly passed two arguments to ensure that the weighted mean of the metrics across all three classes independently, those arguments being average="macro" and zero_division=0. If the model were to fail to predict one of the similar classes during a cross validation fold then the script would output a 0 instead of breaking the whole pipeline.

6 Reproducibility, Github, & Documentation

6.1 GitHub

Our entire project is stored, version-controlled, and hosted on github. This repository is the central source that our team's project revolved around allowing all of our data, visualization, and models to be fully viewable and accessible as a reproducible asset or as a secure backup.

6.2 Reproduce

To reproduce our codebase and run our models presented in this report, users with access to the GitHub repository are able to clone the repository to their local machines. Reproducing our results requires:

- **Environment Setup:** Users should ensure that libraries such as Pandas, Scikit=Learn, and Matplotlib and/or Seaborn are installed in their environment.
- **Correct Execution Order:** Our project is specifically designed to run sequentially in ordered Jupyter Notebook cells. These notebooks should always be run in order, starting with the initial data exploration all the way until the final ML eval notebook. They are labeled P1-4, respectively.
- **Generating Results:** By executing the cells in the final ML notebook in order, the code will automatically filter the dataset, split training and test data, run the cross-validation for the model, then finally output all results including the visualizations.

6.3 Documentation

Our codebase was designed and documented to prioritize easy readability and clinical validity. Notebook documentation was added to each step in each notebook. Using a combination of markdown cells as well

as code comments within code cells we communicated each step of the process for a user to read along highlighting how we prioritized a methodology that is clear and easy to follow for our audience.

References

Fowles, J. B., Rosheim, K., Fowler, E. J., Craft, C., & Arrichiello, L. (1999). The validity of self-reported diabetes quality of care measures. *International Journal for Quality in Health Care*, *11*(5), 407-412.

Mayo Clinic. (n.d.). *Diabetes*. Mayoclinic.org; Mayo Clinic. https://www.mayoclinic.org/diseases-conditions/diabetes/diagnosis-treatment/drc-20371451

Nguyen, B., & Zhang, Y. (2025). A comparative study of diabetes prediction based on lifestyle factors using machine learning. *arXiv preprint arXiv:2503.04137*.

Sapra, A., & Bhandari, P. (2023, June 21). Diabetes. PubMed; StatPearls Publishing. https://www.ncbi.nlm.nih.gov/books/NBK551501/

Tasin, Isfafuzzaman, et al. "Diabetes prediction using machine learning and explainable AI techniques." Healthcare Technology Letters, vol. 10, no. 1–2, 14 Dec. 2022, https://doi.org/10.1049/htl2.12039.

Teboul, Alex. "Diabetes Health Indicators Dataset." Kaggle, www.kaggle.com/datasets/alexteboul/diabetes-health-indicators-dataset/data. [Data set; derived from Centers for Disease Control and Prevention's 2015 Behavioral Risk Factor Surveillance System (BRFSS) Survey].

Thalla, Mohan . "Diabetes Health Indicators Dataset." Kaggle, www.kaggle.com/datasets/mohankrishnathalla/diabetes-health-indicators-dataset/data.

Ullah, Z., Saleem, F., Jamjoom, M., Fakieh, B., Kateb, F., Ali, A. M., & Shah, B. (2022). Detecting high‐risk factors and early diagnosis of diabetes using machine learning methods. *Computational Intelligence and Neuroscience*, *2022*(1), 2557795.